Architektura a vývoj nativního 3D CAD systému na platformě Windows metodou vibecodingu

Vývoj plnohodnotného 3D CAD (Computer-Aided Design) systému na úrovni průmyslových standardů, jakými jsou Autodesk Inventor nebo Siemens Solid Edge, představoval historicky disciplínu vyhrazenou pro desítky specializovaných matematiků a softwarových inženýrů. Příchod metodiky označované jako **vibecoding** – tedy řízení velkých jazykových modelů (LLM) prostřednictvím přirozeného jazyka k produkci reálného kódu při zachování přísné inženýrské disciplíny v oblasti architektury a testování – však dramaticky mění ekonomiku i proveditelnost individuálního vývoje. Pokud je software budován výhradně pro vlastní potřebu bez nutnosti řešit licenční kompatibilitu či autorskoprávní omezení (například striktní virální licence GPLv3 či LGPLv2.1), otevírá se vývojáři možnost přímo zkombinovat nejvýkonnější otevřené výpočetní knihovny světa do jednoho vysoce ergonomického nativního nástroje pro systém Windows.

Paradigma vibecodingu v kontextu výpočetní geometrie

Úspěšná realizace 3D modelovacího programu pomocí AI asistentů vyžaduje pochopení zásadního rozporu mezi pravděpodobnostní povahou jazykových modelů a deterministickou přísností výpočetní geometrie. Pokus o vygenerování celého CAD systému jedním vágním příkazem funguje v praxi jako pověstná „opičí pracka“ (Monkey's Paw) – vede k chaotickému kódu nízké kvality, halucinovaným závislostem, nekonzistentním návrhovým vzorům a rychlému vyčerpání kontextového okna modelu, po němž již ani samotný LLM nedokáže vygenerovaný systém dále udržovat nebo refaktorovat.
Žádný současný jazykový model nedokáže od nuly napsat vlastní stabilní B-Rep (Boundary Representation) geometrické jádro, které by bezchybně počítalo průniky NURBS ploch (Non-Uniform Rational B-Splines), udržovalo Eulerovy topologické operátory a garantovalo uzavřenost těles při zaoblování hran. Cesta k úspěchu proto nespočívá v psaní geometrických algoritmů od nuly, nýbrž v aplikaci **architektury propojování (Glue Architecture)**, která striktně rozděluje systém do čtyř horizontálních architektonických vrstev s obousměrným tokem příkazů a triangulovaných sítí. Toto modulární rozdělení zabraňuje zahlcení kontextového okna jazykového modelu tím, že spodní výpočetní vrstvy přebírá jako kompilované knihovny v C++, zatímco horní orchestrační a prezentační vrstvy přenechává plně generování pomocí AI.
Jak dokazuje moderní open-source projekt **Dune 3D**, který jeho autor Lukas K. sestavil do plně prezentovatelné podoby za pouhé tři měsíce na základě zkušeností s vývojem systému Horizon EDA, funkční 3D CAD lze vybudovat spojením čtyř již existujících stavebních bloků. Úplně spodní fundamentální vrstvu tvoří geometrické jádro provádějící extruze, rotace, booleovské průniky, úkosy a zaoblení, přičemž v otevřeném ekosystému je jedinou plnohodnotnou volbou s podporou formátu STEP a zaoblování hran knihovna **Open CASCADE Technology (OCCT)**, kterou lze pro bleskové booleovské operace a export do 3MF doplnit o síťové jádro **Manifold**. Nad touto základnou leží vrstva matematických řešičů, kde úlohu výpočtu soustav rovnic definujících vzájemné polohy prvků ve 2D i 3D zastává izolovaná knihovna libslvs z programu

SolveSpace, doplněná o engine pro detekci konstrukčního záměru (Live Rules). Třetí vrstvu představuje samotná infrastruktura editoru a orchestrační logika (CAD-as-Code), tedy stavový automat obsluhující interaktivní manipulátory, historii kroků (Undo/Redo), správu dokumentů a serializaci dat. Nejvyšší prezentační vrstvu pak tvoří nativní Windows uživatelské rozhraní a 3D Viewport (postavený na technologiích WinUI 3, WPF nebo Qt6), který zajišťuje vykreslování triangulované geometrie, kamerové transformace (zoom, pan, rotace) a výběr prvků paprskem (raycasting/picking).
Při vývoji metodou vibecodingu přebírá vývojář roli systémového architekta. Nejprve musí přesně zdokumentovat technologický stack, funkční i nefunkční požadavky, datová schémata a testovací případy (včetně okrajových stavů, protože AI generované testy se přirozeně soustředí pouze na bezchybné scénáře tzv. „happy paths“). Samotné generování kódu pak probíhá v přísně izolovaných malých komponentách s průběžným ověřováním a samostatnými commity do verzovacího systému, čímž se z chaotického generátoru stává spolehlivý inženýrský spolutvůrce.

Možnosti vývoje a volba technologického stacku pro Windows

Volba programovacího jazyka a grafického frameworku pro vibecoding CAD aplikace se neřídí pouze surovým výkonem kompilátoru, ale především **hustotou a kvalitou zastoupení daného API v trénovacích datech jazykových modelů** a úrovní abstrakce nad geometrickým jádrem. Níže jsou analyzovány tři nejefektivnější vývojové větve pro nativní prostředí Windows.

Dráha A: Python „AI-Nativní“ stack (PySide6 / Qt6 + build123d / OCP + VTK / OpenGL)

Pro vývoj řízený umělou inteligencí představuje kombinace jazyka Python, nativního vazebního rozhraní **PySide6 (Qt6)** a moderního geometrického frameworku **build123d** zdaleka nejrychlejší a nejstabilnější cestu k cíli.
Hlavním důvodem je fakt, že přímé programování nad C++ jádrem Open CASCADE vyžaduje manipulaci se stovkami nepřehledných tříd (např. TopoDS_Shape, BRepBuilderAPI_Transform, BRepFeat_MakeDPrism) a specifickou správu paměti přes chytré ukazatele Handle(), v nichž jazykové modely často chybují. Naproti tomu knihovna **build123d** (moderní nástupce CadQuery postavený na nízkoúrovňovém Python wrapperu **OCP**) obaluje Open CASCADE do čistého, algebraického a plně pythonického rozhraní s minimálním vnitřním stavem.
V algebraickém režimu build123d lze geometrii skládat pomocí standardních matematických operátorů, kde sčítání (+) reprezentuje booleovské sjednocení, odčítání (-) odečtení objemu, násobení (*) aplikaci prostorových transformací (Pos, Rot, Plane) a operátory @ a % slouží pro okamžité získání souřadnice bodu či tečného vektoru kdekoli na křivce. Výběr topologických prvků (stěn, hran, vrcholů) probíhá přes deklarativní kontejnery ShapeList, které umožňují řetězit filtry podle geometrického typu, plochy nebo osy (např. part.edges().filter_by(Axis.Z)). Protože je tento zápis mimořádně kompaktní a sémanticky čitelný, existují již specializované open-source nadstavby (např. *Text-To-CAD Harness*), v nichž AI agenti generují parametrické 3D modely přímo pomocí build123d.
V této architektuře vykresluje nativní Windows okno knihovna **PySide6** (Qt6 má na Windows plně nativní vzhled a podporu hardwarové akcelerace), zatímco 3D scénu obsluhuje

integrovaný QOpenGLWidget nebo analytický engine **VTK**, do něhož se z OCP posílají ztriangulované sítě. Pro 2D/3D vazbení skic lze připojit C-knihovnu libslvs nebo experimentální numerický řešič solve123d postavený nad knihovnou JAX.

Dráha B: Moderní .NET / C# stack (WinUI 3 nebo WPF + HelixToolkit.SharpDX)

Pokud je prioritou absolutní splynutí s moderním designovým jazykem Windows 11 a silně typovaná architektura MVVM, je optimální volbou platforma **.NET (C#)** v kombinaci s **WPF** nebo **WinUI 3**.
Zásadní výhodou .NET ekosystému je existence knihovny **Helix Toolkit**, konkrétně její hardwarově akcelerované větve HelixToolkit.Wpf.SharpDX a HelixToolkit.WinUI (případně nastupující generace v3 a HelixToolkit.Nex stavějící na moderní architektuře DirectX 11 a Vulkan). Na rozdíl od základního Media3D ve WPF nabízí verze postavená na SharpDX plnohodnotný vlastní 3D engine, graf scény kompatibilní s XAML/MVVM, hardwarový instancing, vykreslování statisíců konstrukčních čar, bodů a billboardů (kótovacích textů), nezávislou průhlednost (Order Independent Transparency - OIT) a vyhlazování hran FXAA. Navíc obsahuje specializovaný balíček HelixToolkit.Wpf.TDxInput a HelixToolkit.Wpf.Input pro okamžitou nativní integraci 3D polohovacích zařízení SpaceMouse.
Úzkým hrdlem čistého C# řešení je napojení na B-Rep jádro Open CASCADE. Oficiální distribuce OCCT sice obsahuje ukázkové projekty jako IE_WPF_D3D, které demonstrují propojení C++ jádra s WPF přes DirectX, avšak vyžadují kompilaci nepřímé zprostředkující vrstvy (proxy knihovny) v **C++/CLI** nebo volání přes P/Invoke. Při vibecodingu v C# lze tuto bariéru překonat několika architektonickými cestami. První možností je využití existujících komunitních či komerčních obálek, jako jsou *OCC C# Wrapper*, *OpenCASCADE.NET* nebo *AnyCAD Rapid SDK*. Druhou cestou je integrace knihovny **ManifoldNET** (oficiální C# NuGet vazba na topologické jádro *Manifold*), pokud se modelář zaměří primárně na booleovské operace s tělesy pro 3D tisk. Třetí a architektonicky nejčistší variantou je **hybridní IPC model (Inter-Process Communication)**, v němž nativní rozhraní a DirectX 11 viewport běží v C# (WinUI 3 / WPF + HelixToolkit), zatímco na pozadí pracuje lokální proces v Pythonu (build123d + OCP), s nímž C# frontend komunikuje přes rychlé lokální sokety nebo roury. Jde o totožný princip předávání zpráv mezi UI a B-Rep jádrem, jaký úspěšně využil projekt *CADmium* nebo jaký doporučují architekti *IfcOpenShell* pro aplikace v .NET 8.

Dráha C: Nativní C++ / Rust nebo Web-Nativní hybrid (Vzory Dune 3D a Plasticity)

Třetí možností je následovat architekturu moderních nezávislých CAD systémů. **Dune 3D** je napsán přímo v C++ s využitím grafického toolkitu GTK4, přímého volání Open CASCADE a upraveného řešiče ze SolveSpace. Komerční, avšak cenově dostupný modelář **Plasticity** (často označovaný jako „CAD pro umělce“ s ovládáním ve stylu Blenderu) a open-source projekt **CADmium** zase ukázaly sílu webových technologií zabalených do nativního kontejneru **Electron**. V případě CADmium obstarával vykreslování Three.js se stavovým frameworkem Svelte (Threlte), zatímco geometrické jádro a sledování historie byly napsány v jazyce **Rust** díky jeho paměťové bezpečnosti, přívětivým chybovým hláškám kompilátoru při refaktoringu a snadné paralelizaci.
Pro individuální vibecoding má však čisté C++ s přímým napojením na OCCT nejvyšší tření,

neboť chybějící hlavičkové soubory v dílčích verzích (např. problémy s kompilací OCCT 7.7.0 oproti 7.7.2) a složité sestavování přes CMake nebo Meson zpomalují rychlou iteraci s AI.

Architektonický stack	Jazyk a UI framework	Geometrické jádro a vazby	Vhodnost pro Vibecoding (LLM)	Nativní Windows UX a výkon
Dráha A: Python AI-Nativní	Python 3.12+, PySide6 (Qt6), VTK / ModernGL	build123d (OCP / Open CASCADE), libslvs / solve123d	**Extrémně vysoká** – algebraický zápis build123d eliminuje halucinace nad C++ API.	Vysoký – Qt6 využívá nativní stylování OS a akcelerovaný OpenGL viewport.
Dráha B1: Čistý .NET / C#	C# (.NET 8/9/10), WPF nebo WinUI 3, HelixToolkit.SharpDX	C++/CLI proxy nad OCCT nebo ManifoldNET + P/Invoke libslvs	**Střední** – LLM skvěle generuje C# a XAML, ale C++/CLI proxy vyžaduje ruční dohled.	**Maximální** – plné využití DirectX 11, XAML/MVVM a nativních Windows API.
Dráha B2: Hybridní C# + Python	C# (WinUI 3 / WPF + HelixToolkit) jako klient; Python jako lokální server	build123d (OCP) + Manifold (manifold3d) na pozadí přes IPC	**Velmi vysoká** – dokonalé oddělení kontextu UI (C#) a geometrie (Python).	**Maximální** – nulové kompromisy v nativním rozhraní i v síle geometrického jádra.
Dráha C: Nativní C++ / Rust	C++20 (Qt6 / GTK4) nebo Rust + Electron (Three.js / Svelte)	Přímé C++ Open CASCADE + interně kompilovaný libslvs	**Nízká až střední** – vysoká verbosita C++ OCCT a složitost build systémů.	Špičkový výpočetní výkon, ale vyšší režie při správě paměti a ladění pádů.

Analýza výkonnostních a provozních dat jednoznačně potvrzuje, že pro vlastní vývoj je optimální **Dráha A (čistý Python s PySide6 a build123d)** nebo **Dráha B2 (hybridní C# frontend s Python geometrickým backendem)**. Tyto přístupy umožňují nejen implementovat postupy synchronní technologie, které v praxi zrychlují cyklus návrhu nových produktů o 30 až 50 % a úpravy sestav o 40 % oproti konkurenčním historickým CAD systémům, ale zároveň kombinovat analytickou přesnost Open CASCADE s extrémní rychlostí síťového jádra Manifold. To v testech na 280 vysokoúrovňových sestavách systému BRL-CAD zvýšilo úspěšnost vyhodnocení z 88,9 % (249 z 280) na 98,9 % (277 z 280) a zkrátilo výpočetní čas ze 7 256 sekund na pouhých 1 044 sekund.

Anatomie jádra: Od parametrické historie k synchronní technologii

Aby vyvíjená aplikace skutečně fungovala s intuitivností programů **Autodesk Inventor** nebo **Solid Edge**, musí její vnitřní architektura správně uchopit reprezentaci těles, řešení vazeb a především způsob, jakým uživatel edituje již vytvořenou geometrii.

Geometrická jádra: B-Rep (Open CASCADE) verzus topologicky

striktní sítě (Manifold)

Průmyslové strojírenské CAD systémy (Inventor, Solid Edge, SolidWorks, Plasticity) nestaví na polygonálních sítích (jako Blender), ale na **hraniční reprezentaci těles (B-Rep)**. V B-Rep struktuře je například krychle definována explicitní topologickou hierarchií, kde těleso (Solid) je ohraničeno skořepinou (Shell) složenou ze šesti stěn (Face), z nichž každá je ohraničena smyčkou (Wire) čtyř hran (Edge), přičemž každá hrana spojuje dva vrcholy (Vertex). Tato topologie je spárována s přesnými analytickými rovnicemi ploch a křivek, jako jsou roviny, válce, kužele, toroidy a NURBS splajny.
Pro dosažení plné kompatibility se strojírenským světem – tedy pro bezchybný import a export normovaných souborů **STEP (ISO 10303)**, přesné analytické zaoblování hran a konstrukční úkosy – je nutné postavit jádro aplikace na **Open CASCADE Technology (OCCT)**. Komerční jádro *Siemens Parasolid*, na němž běží Solid Edge, SolidWorks, Shapr3D i Plasticity a které používá přenosové formáty X_T a X_B bez nutnosti překladu geometrie, není pro individuálního nekomerčního vývojáře volně dostupné ke stažení. OCCT oproti tomu poskytuje kompletní sadu B-Rep algoritmů v otevřeném kódu.
V rámci OCCT obstarává tvorbu a lokální modifikace tvarových prvků balíček BRepFeat, jehož třídy jako BRepFeat_MakePrism, BRepFeat_MakeDPrism (tažení s úkosem), BRepFeat_MakeCylindricalHole a BRepFeat_SplitShape umožňují vytvářet kapsy, výstupky a dělit stěny podél průsečnic přímo na existujícím tělese. Pro technologické úkosy a ofsety slouží balíček BRepOffsetAPI, v němž třída BRepOffsetAPI_DraftAngle modifikuje tvar tělesa aplikací kuželového zešikmení na vybrané stěny (klíčové pro vstřikování plastů a odlévání), přičemž každá stěna přijímá tři parametry: směr vytahování, úhel a neutrální rovinu. Analytické průniky a aproximace řeší moduly Geom2dAPI a GeomAPI, kde Geom2dAPI_InterCurveCurve počítá exaktní průsečíky křivek (včetně vlastních průniků) v toleranci Precision::Confusion(), zatímco GeomAPI_Interpolate konstruuje hladké BSpline křivky se spojitostí C^2 (nebo C^1 při zadaném tečném vektoru) procházející polem bodů. Detekci kolizí a vzdáleností v sestavách pak zajišťuje BRepExtrema, kde BRepExtrema_DistShapeShape počítá minimální exaktní vzdálenost mezi dvěma tělesy pro vyhodnocení dotyku pod prahem d_{min} a BRepExtrema_ShapeProximity využívá předtriangulované sítě pro bleskovou detekci překrývajících se stěn v toleranci d_{ov}.
Pokud však uživatel modeluje extrémně členité díly pro 3D tisk (například mřížkové struktury, zaoblené organické přechody pomocí Minkowskiho součtu nebo pole stovek děr), booleovské operace v Open CASCADE či starším jádře CGAL bývají pomalé a náchylné k selhání na koplanárních či kolineárních hranách. Zde přichází ke slovu knihovna **Manifold** (elalish/manifold), kterou již integrovaly systémy OpenSCAD, Godot Engine i Blender a kterou lze do vlastní aplikace zapojit jako sekundární akcelerační jádro. Manifold pracuje nad trojúhelníkovými sítěmi, ale vynucuje přísnou topologickou podmínku: vrcholy každého trojúhelníku musí být orientovány proti směru hodinových ručiček při pohledu zvenčí a celá množina sítí je uzavřená vůči booleovským operacím. Každý vstup i výstup je tak garantovaně vodotěsné těleso (manifold solid), přičemž knihovna nativně podporuje i generování těles ze vzdálenostních polí (**Signed Distance Fields / SDF** přes level-set funkce namísto algoritmu Marching Cubes) a operace MinkowskiSum a MinkowskiDifference.

Matematický řešič vazeb (libslvs) a ne-modální 3D skicování

Základem parametrického kreslení ve stylu Inventoru je geometrický řešič vazeb (Constraint Solver). Když uživatel nakreslí dvě úsečky a přikáže jim kolmost a konkrétní délku, systém

sestaví soustavu nelineárních algebraických rovnic.
Namísto psaní vlastního numerického řešiče je v architektuře aplikace nejefektivnější použít izolovanou C knihovnu **libslvs** (hlavičkový soubor slvs.h a wrapper lib.cpp), která pochází z programu SolveSpace a představuje jediný plně funkční open-source geometrický řešič vazeb. Komunikace s libslvs probíhá naplněním struktury Slvs_System parametry (param), entitami (entity – body, úsečky, oblouky, normály, pracovní roviny) a vazbami (constraint – vzdálenost, úhel, tečnost, symetrie, bod na přímce nebo kružnici) a následným zavoláním funkce Slvs_Solve(&sys, group_handle).
Knihovna libslvs nabízí kritické vlastnosti pro vysoce interaktivní CAD prostředí. Před spuštěním výpočtu lze pomocí funkce Slvs_MarkDragged označit entitu, kterou uživatel právě táhne myší, načež řešič upraví volné stupně volnosti tak, aby geometrie plynule následovala kurzor při zachování všech definovaných vazeb. Pokud uživatel zadá protichůdné kóty (přeurčená skica), Slvs_Solve při nenulovém ukazateli bad vypočítá a vrátí seznam ID konkrétních konfliktních vazeb v poli ssys->failed, což umožní v uživatelském rozhraní okamžitě zvýraznit problematické kóty červeně. Jak navíc ukázal vývoj Dune 3D, při generování velkého množství 3D rovnic se vyplatí zdrojový kód řešiče přímo propojit a drobně upravit tak, aby triviální rovnice vyřešil nejprve symbolicky před spuštěním numerické iterace, což radikálně zvýší rychlost odezvy při tažení v reálném čase.

Problém topologického pojmenování (TNP) a strategie jeho eliminace

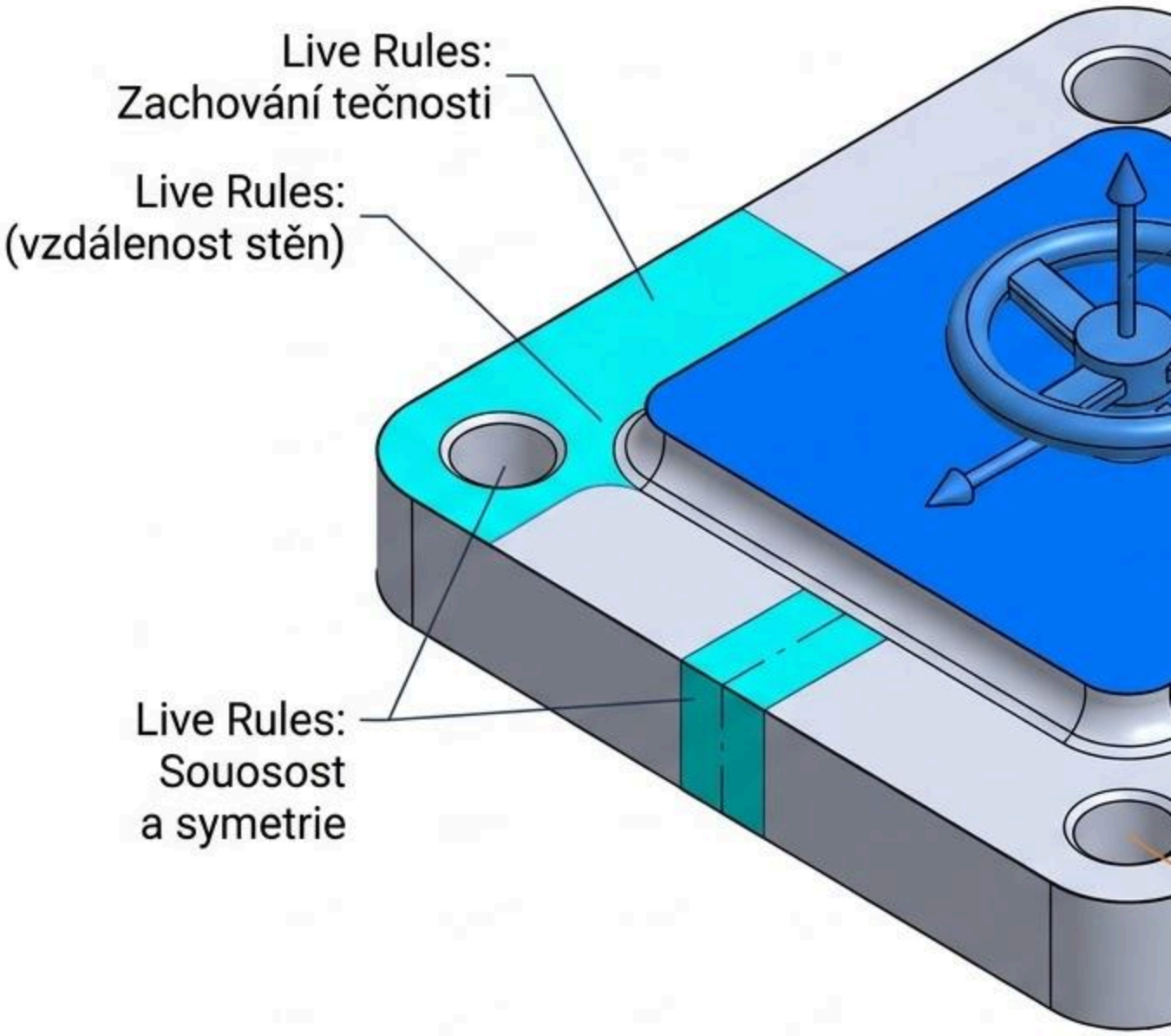
Každý vývojář CAD systému postaveného na historii prvků (History-based modeling) narazí na fundamentální překážku známou jako **Topological Naming Problem (TNP)**. V historickém modeláři vzniká díl postupným vrstvením operací, kde se na stěnu základního kvádru (Face5) umístí skica, z níž se vytáhne válec, a na horní hranu válce (Edge12) se aplikuje zaoblení. Pokud se však uživatel vrátí v historii na začátek a v první skice kvádru srazí roh nebo přidá výřez, jádro Open CASCADE při přepočtu vygeneruje zcela novou instanci TopoDS_Shape, v níž se interní indexy stěn a hran přečíslují. Původní Face5 se náhle stane boční šikmou stěnou, válec „ustřelí“ do prostoru a celé zaoblení havaruje. Právě TNP byl po desetiletí hlavním důvodem frustrace uživatelů systému FreeCAD (než byl pro verzi 1.0 integrován komplexní algoritmus mapování elementů z větve *LinkStage3* od vývojáře *RealThunder*) a zároveň hlavní motivací pro vznik Dune 3D.
Ve vlastní vibecodované aplikaci lze TNP vyřešit kombinací tří architektonických přístupů, které vycházejí ze zásady, že topologie a identita prvků musí být kódována explicitně a nikoliv odvozována z pomíjivých indexů. Prvním přístupem jsou **sémantické (deklarativní) selektory záměru (Intent References)**, jaké používá build123d – namísto ukládání nestabilního číselného indexu Face[4] ukládá historie prvků dotazovací predikát podle geometrické role, například *„rovinná stěna s největším obsahem, jejíž normála směřuje ve směru osy +Z“* (plate.faces().group_by(Face.area)[-1].sort_by(Axis.X)[-1]). I když se počet bočních stěn kvádru změní ze čtyř na dvacet, tento dotaz vždy neomylně vrátí zamýšlenou plochu. Druhým přístupem je **ukládání kontextu sousednosti (Element Map / OCAF Naming)**, tedy využití vestavěného mechanismu topologického pojmenování v aplikačním frameworku Open CASCADE (**OCAF**) ke sledování modifikací tvarů v dokumentu, případně udržování seznamu sousednosti (adjacency list) a prostorového indexu R-stromu (boost::geometry::rtree) souřadnic vrcholů před a po každé změně geometrie. Třetím, nejradikálnějším a uživatelsky nejpřívětivějším řešením je pak opuštění křehkého stromu historie ve prospěch **Synchronního modelování (Synchronous Technology)**, které proslavilo právě zmiňovaný **Siemens Solid**

Edge.

Implementace Synchronní technologie (Synchronous Technology) ze Solid Edge

Tradiční modelování založené na stromu historie nevzniklo z přirozených potřeb konstruktérů, ale jako programátorský kompromis vynucený hardwarovými limity počítačů před třiceti lety. Nutnost předvídat budoucnost při zakládání prvního prvku, „přetáčet strom zpět“ při každé nečekané úpravě (přičemž pozdější prvky dočasně mizí z obrazovky) a opravovat kaskádové selhání rodičovsko-dceřiných závislostí (parent/child) pohlcuje v praxi až **60 % času konstruktéra**.
Synchronní technologie (ST), představená v Solid Edge v roce 2008, tento koncept obrací: neuchovává lineární recept, jak těleso v čase vzniklo, ale pracuje přímo s aktuálním stavem B-Rep geometrie (podobně jako 2D CAD pracuje s přímým výběrem a přesunem čar), přičemž **celý model je řešen najednou**, nikoli sekvenčně prvek po prvku. Prvky uvedené ve stromu modelu (PathFinder) nejsou historickými operacemi, nýbrž pouhými **pojmenovanými výběrovými množinami stěn (face selection sets)**, jejichž pořadí v seznamu lze libovolně měnit bez vlivu na geometrii.
Pro implementaci vlastního „synchronního“ režimu po vzoru Solid Edge je nutné pomocí vibecodingu sestavit čtyři vzájemně provázané mechanismy. Prvním z nich je **3D Volant (Steering Wheel)** jakožto primární grafický manipulátor pro přímou editaci, který se zobrazí okamžitě po kliknutí na libovolnou stěnu nebo výběrovou množinu stěn. Jeho anatomie se skládá z **počátku (Origin)**, **hlavních osových šipek** pro posuv (translaci), **toroidu (kola)** s klouby (bearing knobs) pro rotaci a **pracovní roviny (Tool Plane)** pro volný pohyb ve 2D. Při prvním kliknutí na rovinnou stěnu se zobrazí primární šipka kolmá na tuto stěnu. Zásadní vychytávkou je dvoufázové ovládání, kdy uživatel buď táhne za šipku či toroid a přímo v reálném čase modifikuje geometrii, nebo pomocí modifikačních kláves mění polohu a orientaci samotného Volantu v prostoru: přetažení počátku na hranu zarovná osu Volantu s hranou, Shift + kliknutí na počátek přesune Volant při zachování orientace, Shift + kliknutí na pracovní rovinu otočí orientaci o 180 stupňů a Ctrl nebo Shift + kliknutí na kloub osy natočí Volant v rovině os či směrem k vybranému bodu.
Druhým pilířem je **Engine „Live Rules“ (Pokročilý panel záměru návrhu / Design Intent Panel)**. Samotné přímé modelování (Direct Editing) totiž posouvá pouze explicitně vybranou stěnu, což u složitějších dílů okamžitě rozbije navazující geometrii nebo přidává přepočet do stromu historie. Síla Solid Edge spočívá v tom, že při uchopení stěny Volantem projde engine **Live Rules** v reálném čase graf sousednosti B-Rep tělesa a automaticky předvybere stěny, které udržují konstrukční záměr: souosost válcových ploch, komplanárnost (ležení v jedné rovině), rovnoběžnost, kolmost, osovou symetrii (včetně ploch ležících na obou stranách roviny zrcadlení) a tečnost přilehlých ploch. Pokud uživatel posune stěnu s dírou, souosé prvky se posunou s ní a přilehlé nevybrané stěny se automaticky naklopí tak, aby zachovaly tečný přechod na zaoblení. V uživatelském rozhraní pak plovoucí panel (Solution Manager / Design Intent) umožňuje kterékoliv z těchto automatických pravidel jedním kliknutím dočasně pozastavit, pokud uživatel potřebuje stěnu z vazby vytrhnout.
Třetím mechanismem jsou **3D Řídicí PMI kóty (Driving Dimensions)** a **Živé řezy (Live Sections)**. V synchronním režimu se kóty neukládají v mrtvých 2D skicách, ale umisťují se přímo na 3D hrany tělesa. Zde platí kritické architektonické pravidlo: **kóty se sice vizuálně vytvářejí výběrem hran, ale matematicky řídí vzdálenost a úhel mezi přilehlými stěnami**,

protože hrana v B-Rep topologii neexistuje samostatně, nýbrž vždy vzniká až jako průsečnice dvou stěn. Při změně číselné hodnoty 3D kóty zavolá systém řešič vazeb spolu s pravidly Live Rules a posune příslušné stěny. Pro rotační a tvarově složitá tělesa, kde je obtížné chytat obrysové hrany, vygeneruje systém **Živý řez (Live Section)** – 2D rovinný řez procházející 3D tělesem, jehož 2D čáry přímo řídí 3D plochy modelu.
Čtvrtým prvkem je **Hybridní modelování (Synchronous + Ordered)**. Jak ukazují osvědčené postupy ze Solid Edge (tzv. pravidlo 75/25 pro strojírenské díly vs. pravidlo 25/75 pro stylizované spotřební produkty), synchronní přístup exceluje na prizmatických součástech a při okamžitých úpravách importovaných cizích dat od dodavatelů bez nutnosti jejich rekonstrukce, ale je méně vhodný pro složité tažené přechody. Ideální architektura proto kombinuje oba světy v jednom souboru i sestavě: prizmatické konce např. ramene nápravy se vymodelují, zkopírují a napolohují Volantem v synchronním režimu, a následně uživatel jedním kliknutím přepne do sekvenčního („Ordered“) režimu a spojí je přechodovým tažením (Loft).
Matematicky lze lokální přímou modifikaci rovinné stěny v B-Rep jádře (bez přepočtu historie) realizovat tak, že pro vybranou stěnu F_i s rovnicí roviny \mathbf{n}_i \cdot \mathbf{x} + d_i = 0 a transformační matici Volantu \mathbf{T} \in SE(3) (získanou v OCCT přes gp_Trsf) systém nejprve identifikuje množinu vázaných stěn \{F_k\} splňujících predikáty Live Rules (např. \mathbf{n}_k \times \mathbf{n}_i = \mathbf{0} a d_k = d_i pro komplanární stěny). Následně se pomocí analytického prodloužení sousedních ploch (v OCCT přes BRepFeat nebo přepočtem průsečnic sousedních nekonečných ploch Geom_Surface a znovuzastřižením hranic TopoDS_Wire) zkonstruuje upravená B-Rep skořepina.



Klíčové funkce, ergonomie a AI-nativní vychytávky pro vlastní CAD

Při stavbě aplikace pro vlastní potřebu není vývojář svázán korporátní zpětnou kompatibilitou a může zkombinovat nejrychlejší pracovní postupy ze **Solid Edge**, **Inventoru**, **Plasticity** a **Dune 3D**, obohacené o funkce, které umožňuje pouze přímé zapojení AI do jádra programu.

Moderní interakční vzory: Kombinace Solid Edge, Plasticity a Dune 3D

Tradiční CAD systémy trpí přebujelými pásy karet (Ribbons), roztříštěností pracovních prostředí (jako je matoucí oddělení modulů *Part* a *Part Design* ve FreeCADu, kde chybná volba na začátku nutí uživatele smazat díl a začít znovu) a nutností neustále přepínat mezi izolovaným 2D režimem náčrtu a 3D prostorem.
Moderní intuitivní rozhraní by mělo především opustit modální 2D skicář a zavést **nefiltrované 3D skicování a vazbení extruzí po vzoru Dune 3D**. Namísto „uzamčení“ uživatele do ploché 2D skici probíhá kreslení čar a Beziérových křivek přímo ve 3D prostoru, přičemž uživatel si jedním příkazem může zvolit pracovní rovinu (set workplane), ale délku vytažení (extruze) nemusí zadávat jen pevným číslem – může ji přímo ve 3D zavazbit k jinému bodu, stěně nebo referenčnímu STEP modelu v prostoru pomocí 3D řešiče libslvs. Samotné skici by navíc měly podporovat kopírování a vkládání (Copy-Paste) i mezi různými dokumenty, rozdělení konstrukční geometrie do nezávisle skrývatelných **vrstev (Geometry Layers)** a funkci adaptivního přenosu obrysů (**Carbon Copy**) s možností synchronizace na vyžádání při změně zdrojové skici.
Pro bleskovou dostupnost nástrojů je vhodné nahradit klasické roletové nabídky **vyhledávací paletou příkazů (Command Palette)** vyvolanou stiskem **mezerníku (Spacebar)** po vzoru Dune 3D, Horizon EDA a Plasticity. Nejčastější operace pak zpřístupní kruhové **radiální menu** (inspirované Blenderem a Plasticity) a kontextový **Quick Bar** objevující se přímo u kurzoru při výběru prvku (jako v Solid Edge), což zcela eliminuje nutnost jezdit myší k okrajům obrazovky.
Při práci s hranami a plochami by aplikace měla zpřístupnit **inteligentní adaptivní zaoblování (Fillets)**, které se v reálném čase přizpůsobuje okolní geometrii a udržuje hladkou spojitost křivosti G^2 (vzor Plasticity) bez kolapsů typických pro starší open-source nástroje. Pro plynulou prostorovou navigaci, kdy levá ruka plynule otáčí a přibližuje model, zatímco pravá ruka ovládá 3D Volant myší, je pak nezbytná nativní podpora **3Dconnexion SpaceMouse** (implementovatelná přes otevřenou alternativu ovladačů jako v Dune 3D nebo přes balíček HelixToolkit.Wpf.TDxInput).

Obousměrná synchronizace GUI a kódu (Bi-directional CAD-as-Code)

Největší architektonickou výhodou vlastního CADu postaveného na knihovně **build123d** je možnost smazat hranici mezi klikacím GUI (jako Inventor či Solid Edge) a programovým modelováním (jako OpenSCAD).
V této architektuře je nativním souborovým formátem aplikace přímo **čistý skript v jazyce Python (build123d)**. Když uživatel v 3D okně vybere horní stěnu, klikne na 3D Volant a vytáhne z ní válec nebo srazí hranu vývrtu, aplikace na pozadí nejen upraví B-Rep geometrii, ale automaticky zapíše do postranního editoru řádek čitelného deklarativního kódu, například bore = part.faces().filter_by(GeomType.CYLINDER).filter_by(lambda f: f.radius == 2) a part = chamfer(bore.edges(), 0.2). Při přepočtu skriptu přitom kamera v editoru zůstává naprosto stabilní, jak je tomu v moderních verzích ManifoldCADu.
Tento obousměrný (bi-directional) přístup přináší dvě zásadní funkce. První je **integrovaný AI kopilot přímo nad 3D scénou**. Protože celý stav modelu je reprezentován kompaktním kódem build123d bez skrytého binárního stavu, lze do aplikace vložit textové pole napojené na LLM

API. Když uživatel v 3D viewportu označí myší stěnu (čímž předá její selektor plate_face do kontextu) a napíše v přirozeném jazyce pokyn k vytvoření mřížky zahloubených děr, AI okamžitě vygeneruje příslušný build123d výraz plate -= Plane(plate_face) * GridLocations(13, 3, 2, 2) * CounterSinkHole(.5, 1, 2), provede jej a zobrazí výsledek v reálném čase. Druhou výhodou je **skutečné pole polí (Pattern of Patterns) a polární skicování**. Zatímco tradiční open-source GUI modeláře (jako FreeCAD) často selhávají, pokud chce uživatel vytvořit obdélníkové pole prvků a toto pole následně zrcadlit nebo polem opakovat (což Autodesk Inventor či SolidWorks zvládají bez problémů), v architektuře podložené kódem build123d jde o přirozenou kombinaci polárních souřadnic, křivek se zaoblenými rohy a vnořených lokátorů, které lze libovolně skládat.

Specializované nástroje pro 3D tisk, elektroniku a přímou výrobní pipeline

Pro konstrukční, prototypovací a dílenské účely se do programu hodí integrovat specializované generátory a přímé mosty do navazujících nástrojů, které šetří hodiny rutinní práce.
V oblasti elektroniky jde především o **ECAD/MCAD asistenta pro návrh krabiček kolem desek plošných spojů**, což byl hlavní důvod vzniku Dune 3D a oblíbený workflow komunity build123d + KiCad. Aplikace umožní naimportovat STEP model osazené desky plošných spojů jako referenci a přímo k jejím součástkám a montážním otvorům ve 3D zavazbit stěny krabičky, sloupky pro mosazné zálisky (inserts and bosses), zaklapávací spoje (snap-fits) či ohebné panty (living hinges).
Pro okamžitou výrobu na 3D tiskárně by aplikace měla zcela nahradit ruční export zastaralých STL souborů (které nejsou indexované a nezaručují vodotěsnost geometrie) přímým tlačítkem **„One-Click Print“ do PrusaSliceru, OrcaSliceru nebo Bambu Studia**. Na pozadí program vyexportuje topologicky seřazený soubor **3MF** (nebo přesný STEP) a pomocí systémového volání spustí konzolový binární soubor prusa-slicer-console.exe nebo CLI rozhraní Bambu Studia s parametry --load_settings, --load_filaments a --load_printers (případně využije profily vložené přímo v 3MF kontejneru). Pro vizuální prezentaci a fotorealistické renderování, které není smyslem samotného CAD jádra, je pak ideální implementovat **Živý most do Blenderu (Live Blender Bridge po vzoru Plasticity)** – síťový konektor, který v reálném čase přenáší změny sítě, materiály a hierarchii objektů přímo do otevřené scény v Blenderu, případně exportuje modely ve formátu glTF.

Kategorie funkce	Konkrétní „vychytávka“ (Feature)	Inspirační vzor	Technická realizace ve vibecoded architektuře
Přímá manipulace	**3D Volant (Steering Wheel)** + **Quick Bar**	Siemens Solid Edge	Vlastní 3D gizmo ve viewportu (HelixToolkit / VTK) generující matice gp_Trsf pro vybrané TopoDS_Face.
Záměr návrhu	**Live Rules Engine** (automatická symetrie, souosost, tečnost)	Siemens Solid Edge	Průchod grafu sousednosti B-Rep stěn před aplikací transformace; porovnání normál a os válců v toleranci.
Parametrika bez	**3D Řídicí PMI kóty** a	Solid Edge / Plasticity	Kóty vykreslené jako

Kategorie funkce	Konkrétní „vychytávka“ (Feature)	Inspirační vzor	Technická realizace ve vibecoded architektuře
stromu	**Živé řezy (Live Sections)**		3D billboardy nad hranami, které předávají rovnice vzdálenosti přilehlých stěn do libslvs.
Ergonomie skicování	**Ne-modální 3D skicování** a vazbení délky extruze ve 3D	Dune 3D / SolveSpace	Odstranění 2D modálního režimu; napojení všech 3D bodů a vektorů tažení přímo do 3D skupiny Slvs_Solve.
Rychlost ovládání	**Command Palette (Mezerník)** + **Radiální menu**	Plasticity / Blender / Dune 3D	Globální ne-modální vyhledávací overlay okno filtrované přes fuzzy-match nad registrem všech CAD příkazů.
AI & Automatizace	**Obousměrný GUI ↔ Python (build123d) generátor**	build123d / Text-to-CAD	Každá akce v GUI modifikuje AST strom Python skriptu; vestavěný LLM chat upravuje tentýž skript na základě výběru.
Výrobní pipeline	**PCB Enclosure Wizard** + **1-Click 3MF Slicer & Blender Bridge**	Dune 3D / Plasticity / PrusaSlicer	Analýza děr v importovaném STEP modelu PCB; přímé volání prusa-slicer-console.exe / Bambu CLI přes podproces.

Strategický plán implementace a orchestrace AI agentů

Aby vývoj pomocí vibecodingu neskončil u neudržovatelného prototypu, je nezbytné postupovat v přísně oddělených fázích, kde každá etapa má jasně definované rozhraní, správu verzí a automatizované testy.

První etapa spočívá ve vybudování **bezhlavého geometrického jádra (Headless Geometry Engine)**. Vývojář nejprve nechá AI vygenerovat čistě výpočetní modul v Pythonu (nad build123d / OCP a manifold3d) bez jakéhokoliv grafického rozhraní. Tento modul musí přijímat strukturované příkazy, provádět konstrukci B-Rep těles, aplikovat sémantické selektory stěn či hran a vracet ztriangulované pole vrcholů, normál a mapu ID topologických prvků pro následný výběr myší. Protože objekty Open CASCADE nejsou přímo serializovatelné standardním modulem pickle, musí architektura od počátku ukládat stav do čitelných skriptů build123d nebo souborů BREP/STEP a ověřovat vodotěsnost výstupů sadou jednotkových testů (inspirovaných

např. Apache-2.0 licencovanou testovací sadou projektu Manifold).
Druhá etapa zahrnuje **integraci řešiče vazeb (libslvs) a grafu sousednosti (Live Rules)**. V tomto kroku se připojí kompilovaná C-knihovna libslvs a AI dostane za úkol napsat zapouzdřený překladač mezi entitami 3D skici a strukturami Slvs_System, včetně testů pro detekci přeurčených vazeb (ssys->failed) a plynulé tažení kurzorem (Slvs_MarkDr[span_109](start_span)[span_109](end_span)[span_112](start_span)[span_112](end_span)agged). Souběžně se nad TopoDS_Shape a třídami BRepExtrema implementuje topologický skener, který před každou přímou úpravou vyhledává v modelu komplanární, souosé, tečné a symetrické stěny pro modul Live Rules.
Ve třetí etapě přichází na řadu **nativní 3D Viewport a interaktivní manipulátor (Picking & Steering Wheel)**. Teprve nad plně otestovaným výpočetním jádrem se vygeneruje okenní aplikace v PySide6 nebo WinUI 3 / WPF s engine HelixToolkit.SharpDX (při pečlivém dodržení pravotočivého souřadnicového systému a orientace trojúhelníků). Klíčovým úkolem pro LLM je zde přesné mapování paprsku myši (Raycasting) na konkrétní FaceID a EdgeID z geometrického jádra a vykreslení 3D manipulátoru **Steering Wheel**, který transformuje pohyb myši a klávesové modifikátory (Shift, Ctrl) na transformační matice předávané do synchronního řešiče.
V závěrečné čtvrté etapě se nad hotovým editorem vystaví **ergonomická nadstavba, AI kopilot a externí výrobní mosty**. Do rozhraní se doplní mezerníková Command Palette, radiální menu, kontextový Quick Bar a 3D řídicí PMI kóty. Následně se propojí postranní editor živého kódu build123d s API jazykového modelu pro generování a úpravy tvarových prvků textem a připojí se konzolové exportéry do formátu 3MF pro PrusaSlicer/Bambu Studio a živý soketový most do Blenderu.
Dodržením této čtyřfázové modulární dekompozice se veškerá složitost výpočetní geometrie deleguje na průmyslově ověřené kompilované knihovny (Open CASCADE, Manifold, SolveSpace), zatímco jazykový model operuje výhradně v doméně, v níž je nejsilnější – při tvorbě nativního uživatelského rozhraní, stavové orchestrace a čitelného deklarativního kódu. Výsledkem je plnohodnotná nativní Windows CAD aplikace šitá přesně na míru, která kombinuje bleskovou přímou editaci Solid Edge s neomezenou parametrickou silou moderního skriptovacího inženýrství.

Citované zdroje

1. AI For Luddites: VibeCoding - by Shan McAdoo - Medium, https://medium.com/@r19slr/vibecoding-for-luddites-9c1a419c9f8d 2. Vibe coding - Wikipedia, https://en.wikipedia.org/wiki/Vibe_coding 3. Vibecoding and AI in Software Development – Trend or ... - Moravio, https://www.moravio.com/blog/vibecoding-just-a-trend-or-a-serious-method 4. dune3d/dune3d: 3D CAD application - GitHub, https://github.com/dune3d/dune3d 5. JacobStoren/SolveSpaceLib - libslvs · GitHub, https://github.com/JacobStoren/SolveSpaceLib 6. Read a software engineering blog if you think vibe coding is the future, https://www.reddit.com/r/vibecoding/comments/1kprxpl/read_a_software_engineering_blog_if_you_think/ 7. Topology in CAD: Manifoldness, Genus, and the Earliest B-Rep, https://novedge.com/blogs/design-news/design-software-history-topology-in-cad-manifoldness-genus-and-the-earliest-b-rep-formalisms 8. Vibe Coding - Martin Fowler, https://martinfowler.com/bliki/VibeCoding.html 9. CADmium: A Local-First CAD Program Built for the Browser, https://mattferraro.dev/posts/cadmium 10. Introducing Dune 3D: A New Open-Source CAD Tool for Simplified, https://www.fabbaloo.com/news/introducing-dune-3d-a-new-open-source-cad-tool-for-simplified-

3d-design 11. elalish/manifold: Geometry library for topological robustness - GitHub, https://github.com/elalish/manifold 12. Releases · elalish/manifold - GitHub, https://github.com/elalish/manifold/releases 13. Parametric vs Direct Modeling: Which Saves More Time? - SimuTecra, https://simutecra.com/blogs/parametric-vs-direct-modeling-time-comparison 14. build123d - PyPI, https://pypi.org/project/build123d/ 15. Displaying CadQuery data in a QT5 window in Python - Stack Overflow, https://stackoverflow.com/questions/67772683/displaying-cadquery-data-in-a-qt5-window-in-python 16. HelixToolkit.Wpf 3.0.0-ci0001 - NuGet Gallery, https://www.nuget.org/packages/HelixToolkit.Wpf/3.0.0-ci0001 17. The problem with vibe coding is nobody wants to talk about ... - Reddit, https://www.reddit.com/r/vibecoding/comments/1o547xp/the_problem_with_vibe_coding_is_nobody_wants_to/ 18. The hidden cost of vibecoding - by Boris Haviar - Medium, https://medium.com/@boris.haviar/the-hidden-cost-of-vibecoding-e8e76870f628 19. uhub-awesome-python/README.md at master - GitHub, https://github.com/ip2location/uhub-awesome-python/blob/master/README.md 20. A curated list of awesome Python frameworks, libraries and software., https://github.com/uhub/awesome-python 21. pythonocc-demos/examples/core_modeling_bool_demo.py at master, https://github.com/tpaviot/pythonocc-demos/blob/master/examples/core_modeling_bool_demo.py 22. Source code for cadquery.occ_impl.shapes, https://cadquery.readthedocs.io/en/stable/_modules/cadquery/occ_impl/shapes.html 23. Open CASCADE Technology 7.2.0 - documentation.help, https://documentation.help/Open-Cascade/documentation.pdf 24. OpenCasCade official development document translation (4), https://www.caxkernel.com/8164.html 25. Tips, Best Practices and FAQ - Build123d, https://build123d.readthedocs.io/en/latest/tips.html 26. How I Built a Parametric 3D Model Generator Using Build123d, https://www.instructables.com/How-I-Built-a-Parametric-3D-Model-Generator-Using-/ 27. Text To Cad Harness (Grade A) - Claude Skill | Skills Directory, https://www.skillsdirectory.com/skills/reason-machines-text-to-cad-harness 28. Top 23 Pyqt Open-Source Projects - LibHunt, https://www.libhunt.com/topic/pyqt 29. Build123d: A Python CAD programming library - Hacker News, https://news.ycombinator.com/item?id=47567242 30. How can I obtain the correct IfcOpenShell C# DLLs (IfcGeomSharp, https://github.com/IfcOpenShell/IfcOpenShell/discussions/7988 31. HelixToolkit.WinUI 2.27.3 - NuGet, https://www.nuget.org/packages/HelixToolkit.WinUI 32. Package HelixToolkit.Wpf.SharpDX - GitHub, https://github.com/orgs/helix-toolkit/packages/nuget/package/HelixToolkit.Wpf.SharpDX 33. Packages · Helix Toolkit - GitHub, https://github.com/orgs/helix-toolkit/packages 34. HelixToolkit.Wpf.SharpDX 3.0.0-ci0001 - NuGet, https://www.nuget.org/packages/HelixToolkit.Wpf.SharpDX/3.0.0-ci0001 35. Is it possible to migrate to HelixToolKit.WPF.SharpDX from, https://stackoverflow.com/questions/55002747/is-it-possible-to-migrate-to-helixtoolkit-wpf-sharpdx-from-helixtoolkit-wpf-in-c 36. Computer Aided Engineering [计算机辅助工程] - 崔济东的博客, http://www.jdcui.com/?page_id=24339 37. cam · GitHub Topics, https://github.com/topics/cam?l=c%23&o=desc&s=stars 38. fel88 - NuGet Gallery, https://nugetprodusnc-northcentralus-01.regional.azure-api.net/profiles/fel88 39. Plasticity review: an easy-to-use 3D modelling and CAD package for, https://www.creativebloq.com/3d/3d-software/forget-what-you-know-about-cad-plasticity-has-ma

de-3d-modelling-fun-again 40. Still can't decide on which modeling software to learn. What would,
https://www.reddit.com/r/3Dprinting/comments/1qxph2f/still_cant_decide_on_which_modeling_software_to/ 41. Dune 3D: Open Source 3D Parametric Modeler From The Maker Of,
https://hackaday.com/2024/05/05/dune-3d-open-source-3d-parametric-modeler-from-the-maker-of-horizon-eda/ 42. Designcenter Solid Edge - SIS - Spatial Integrated Systems,
https://sisinc.org/solid-edge/ 43. Users of Manifold · elalish manifold · Discussion #340 - GitHub,
https://github.com/elalish/manifold/discussions/340 44. Siemens PLM Chapters 1 3 Solid Edge Synchronous Technology,
https://www.scribd.com/document/322073738/Siemens-PLM-Chapters-1-3-Solid-Edge-Synchronous-Technology-eBook-Tcm1023-247523 45. Synchronous technology | Siemens,
https://www.siemens.com/en-us/technology/synchronous-technology/ 46. Solid Edge | 3D Design, Simulation & Manufacturing, https://team-eng.com/our-products/solid-edge/ 47. What are the differences between parasolid, STEP and IGES,
https://www.reddit.com/r/SolidEdge/comments/120g4m9/what_are_the_differences_between_parasolid_step/ 48. Learn the Solid, Not the File: Canonical Inputs for Neural Networks,
https://arxiv.org/pdf/2609.11573 49. OpenCASCADE 基础（转） - 格美格美- 博客园,
https://www.cnblogs.com/mrliu0515/p/16211012.html 50. Open CASCADE学习 |BRepOffsetAPI_DraftAngle 原创 - CSDN博客,
https://blog.csdn.net/T20151470/article/details/137728157 51. APLAN: open assembly planning framework in FreeCAD,
https://lirias.kuleuven.be/retrieve/d23ef361-9a85-4d87-8b16-aa43a45f2d3c 52. How does it compare against https://github.com/elalish/manifold in,
https://news.ycombinator.com/item?id=49083611 53. Manifold Library · elalish/manifold Wiki - GitHub, https://github.com/elalish/manifold/wiki/Manifold-Library 54. guaranteed manifold meshes in C++ - Hysen Labs, https://hysenlabs.com/projects/elalish-manifold 55. solvespace/src/slvs/lib.cpp at master - GitHub,
https://github.com/solvespace/solvespace/blob/master/src/slvs/lib.cpp 56. solvespace/include/slvs.h at master - GitHub,
https://github.com/solvespace/solvespace/blob/master/include/slvs.h 57. FreeCAD's topological naming problem is (officially) history - Ondsel,
https://www.ondsel.com/blog/toponaming-problem-is-history/ 58. Dune3D | Hackaday,
https://hackaday.com/tag/dune3d/ 59. The color problem of Boolean operation | OCCT Forum Archive, https://occt3d.com/dev/content/color-problem-boolean-operation/ 60. Dune 3D - parametric 3D CAD - Off-Topic - Haiku Community,
https://discuss.haiku-os.org/t/dune-3d-parametric-3d-cad/14984 61. Topological Naming · realthunder/FreeCAD_assembly3 Wiki · GitHub,
https://github.com/realthunder/FreeCAD_assembly3/wiki/Topological-Naming/f613e390f149d73c454d018933b51ade7309c9c0 62. Sketcher Development - Integration of Extensions,
https://devtalk.freecad.org/t/sketcher-development-integration-of-extensions/4615 63. Examining Solid Edge Synchronous Technology from a Parametric,
https://www.digitalengineering247.com/article/examining-solid-edge-synchronous-technology-from-a-parametric-point-of-view 64. Working in Traditional and Synchronous Mode Using Solid Edge, https://www.plm.automation.siemens.com/de_ch/Images/18130_tcm782-84654.pdf 65. Solid Edge with synchronous technology - Digicad,
https://www.digicad.fr/pdf/Solid-Edge-Synchronous-Technology-ebook-61045-tcm1023-254657.pdf 66. Hybrid modeling in Solid Edge with syncronous and ordered modes,
https://blogs.sw.siemens.com/solidedge/hybrid-modeling-in-solid-edge-with-syncronous-and-ord

ered-modes/ 67. Steering Wheel Advanced Techniques | Designcenter Solid Edge, https://blogs.sw.siemens.com/solidedge/steering-wheel-advanced-techniques/ 68. Review: Solid Edge ST8 - DEVELOP3D, https://develop3d.com/reviews/review-solid-edge-st8-synchronous-tectnology-design-engineering-siemens/ 69. SpaceClaim 2012 - Index of /, https://forum.logicad.ro/Spaceclaim/SpaceClaim2012_SP1_UsersGuide.pdf 70. Review: Solid Edge with Synchronous Technology 4, https://www.digitalengineering247.com/article/review-solid-edge-with-synchronous-technology-4 71. Plasticity - 3D Modeling Tool - UI UX Showcase, https://uiuxshowcase.com/resources/plasticity-3d-modeling-tool/ 72. Dune 3D, https://dune3d.org/ 73. Weekly recap — 29 October 2024 - Libre Arts, https://librearts.org/2024/10/week-recap-29-oct-2024/ 74. dune3d-docs - GitHub, https://github.com/dune3d/dune3d-docs 75. New parametric CAD app Dune 3D - Maker Forums, https://forum.makerforums.info/t/new-parametric-cad-app-dune-3d/88582 76. Newbie wants to try Build123d after openscad with bosl2 library, https://www.reddit.com/r/build123d/comments/1g9g8yh/newbie_wants_to_try_build123d_after_openscad_with/ 77. Plasticity - CAD for artists - Fountn, https://fountn.design/resource/plasticity-cad-for-artists/ 78. Automate Creation of G-code files from Prusa Slicer? - Reddit, https://www.reddit.com/r/prusa3d/comments/qwgnzg/automate_creation_of_gcode_files_from_prusa_slicer/ 79. Static and Nonlinear FEA – Plastics and Metals, https://hightechcom.de/pages/static-and-nonlinear-fea-plastics-and-metals 80. CLI cannot apply external presets - Bambu Studio, https://forum.bambulab.com/t/cli-cannot-apply-external-presets/197842 81. Modelist Features: Library, ZIPs & Duplicate Finder, https://modelist.app/features 82. Command Line Guide - SuperSlicer, https://docs.superslicer.org/advanced-usage-guides/cmd-line-guide/ 83. Command Line Interface · prusa3d/PrusaSlicer Wiki - GitHub, https://github.com/prusa3d/PrusaSlicer/wiki/Command-Line-Interface/7cc20b4b1b317f0a16bde1858769da0620392036 84. Is Plasticity Enough, or Should I Learn Fusion 360, NX, or SolidWorks?, https://www.reddit.com/r/Plasticity3D/comments/1u6f4ae/choosing_a_longterm_cad_path_is_plasticity_enough/ 85. Caching Build123D objects? - Reddit, https://www.reddit.com/r/build123d/comments/1gnvd4n/caching_build123d_objects/